

Why Agentic Systems Need Ontologies — Frank Coyle, UC Berkeley

Transcribed by [TurboScribe](#). [Go Unlimited](#) to remove this message.

Okay, we're going to launch here. So my name is Frank Coyle, I'm an educator, I'm teaching at Berkeley now, I've been doing this computer science stuff for, oh, 30, 35 years, and right now it's kind of a critical time for the poor computer science students. It used to be the only game in town, degree was a guaranteed job, and now, thanks to AI, it's not.

But then again, 5,000 people are here, so AI and agents seem to be the way to go. So the question is, how do we leverage this new universe that we are moving quickly into? And so I want to talk about how agents and ontologies, big word, fit together. But before I do that, I wanted to give you my educational philosophy, and this comes from someone called Sister Corita Kent, and it was made popular by John Cage, who is an avant-garde musician, and you've got to think about this a little bit.

Nothing is a mistake. There is no win, there's no fail, there's only make. And more and more today, that's what's important.

Get down and make stuff, and that's how you're going to learn, not by necessarily reading. I'm also a big fan of writing. My early career was in neuroscience, I'm kind of coming back into it now that agentic AI is bringing kind of cognitive science back.

But engage your senses. Get a notebook, get a pen, a pencil, draw pictures, write stuff down. Just don't type, because when you're typing, your brain is thinking about the letters on the keyboard.

When you're writing in a book, your whole brain, all your sensory systems are engaged, and you're going to learn faster that way. Okay, on to our talk, agents and ontology. So there are two lineages here, and I want to talk about both, give you a little philosophical background.

Agents, when did we start talking about agents? Well, it goes back to the early, initial days of AI. People like John McCarthy, Selfridge, Marvin Minsky, Society of Mind, people started thinking about the fact that this new computing technology was going to lead us into some kind of artificial intelligence, which is a term that came in 1956, when all these characters got together and tried to figure out where the future was going. And the concept of an agent finally evolved, things that perceive and decide and then act, and that's what we're seeing now.

Now what about ontologies? Well, it turns out, ontologies are not that new. Okay, it was actually Aristotle who first came up with the concept of, we need a philosophy of being, like, ooh, kind of heavy, but came up with categories of being, and this kind of relates to what people are doing now with graph databases and knowledge representation. And there were a couple of other

people who kind of formalized it, Von Quine was a philosopher, and then this guy Gruber, 1993.

And I think this captures what knowledge and graph technology really represents. It is a formal specification of a shared conceptualization, and that's what we want to give to our agents. We want to give them our conceptualization of the universe, our universe, our domains.

Okay, and now what's happening is you're getting the convergence of something that is probabilistic, the agents, the LLMs, with the more formal representations that you have with ontologies. And so this term is now being used, you're hearing this a lot, neuro-symbolic AI. Sounds pretty fancy, but it's really neural networks tied into symbolic AI, which rule-based systems come under that category, as do the knowledge graphs that we're assembling.

And so what I like to argue is that neuro-symbolic AI sort of represents a way to keep the LLM on its guardrails, because LLMs are by nature probabilistic. People worry about hallucinations, but that's the feature, that's actually a feature of large language models. It's who we are.

We hallucinate in a way, we imagine things that may not exist, and then we turn them into reality. And that's what large language models do in a way. So let's just quickly overview what ontologies are.

They're not complicated, they're basically a representation of entities and their relationships to other entities, and these entities have properties. And this whole concept of graph databases arose when people began to realize that relational databases, sticking data into tables, was too restrictive. You wanted to add something new to a relational database, so you had to add a new column.

Man, then you have to redo the whole structure. With a graph database, you can just attach another item, you can just attach a property, you can attach a relationship. So the question often arises, okay, I get it, I need to have an ontology to represent in a formal way what my organization is doing.

How do I do it? Okay, there are a couple of ways you can approach it. You can have a top-down approach or a bottom-up approach. Top-down approach is you get the experts together and they sit down and analyze the domain, come up with the entities, what do we have? We have purchase orders, we have customers, we have customer representatives, and we're going to structure them, they have properties, these are the relationships.

Okay, that's one way. And this models what we were doing back in the 80s when I was involved in expert systems. Everybody thought expert systems was the way to do AI.

Symbolic AI was the way to go. Companies rose, millions of dollars were spent. The Japanese created this future world project in the late 80s.

People in America, my son was taking Japanese in school because of these expert systems, but

they couldn't scale. They couldn't scale, and then we went into a kind of AI winter. Where did neural networks come from? Neural networks were put out there in the 60s, but they couldn't scale because we didn't happen to have NVIDIA, who was off making GPUs to make reality of the video games fantastic.

And then someone said, let's turn these things over to the neural networks, and of course, that's kind of why we're here now. So the other way you can, that people are adding to or creating ontologies is from the bottom up. For example, customer reactions.

What are the things the customers are involved in? Hey, these entities, these relationships, let's add this to our ontology. Let's add this information to the graph. Now as a help, it's helpful to be aware that there are existing taxonomies that people have been working on for the last 15 to 20 years.

Things like schema.org, which has a whole set of terms and relationships, so you don't have to reinvent the wheel. In fact, it's to your advantage to use some of these ontologies. FOAF, a friend of a friend for modeling social networks.

The Dublin Core, which was an early attempt to come up with terms for describing research papers and books and so forth. So there's a whole series of things. In fact, Wikipedia is based on an ontology called DBpedia.

So when you do a search on Wikipedia, it's looking things up in its giant graph database. So this stuff has been out there underlying a lot of what we already do. So take advantage of these things that already exist.

Okay, now, what do you do when you build your ontology? Okay, so what? I know what these entities are, I know what their relationships are, they have properties. How can I do anything with them? Well, there are other augmenting technologies, auxiliary technologies. Things that we call, things like RDFS, which is a technology, and OWL, which I'll talk more about.

So these have, these kind of sit over to the side of your graph. So I'm not going to talk about, I mean, ontology is a big word and it's often confusing and used in many ways. But think you have a graph data structure, okay, and you have the entities and relationships, but you want to apply some control over them.

Or you want to be able to make inference over them. So for example, there is some terms in this technology called RDFS, domain and range. So if I say, teaches has a domain of teacher.

That means if I say, Bob teaches scooter in my text, I can infer that Bob is a teacher. And if I say, all teachers are persons, then this statement lets me know, if I say, Bob teaches scooter, now I know Bob is a person, Bob is a teacher. What about scooter? If I say, teaches has a range of student, that means the right side of the verb, then scooter is a student.

And now I have this extra information into my system. OWL also has a series of properties that allow you to make some inferences. So a transitive property.

Transitive property says, if Sue is an ancestor, like ancestor is a transitive property. If Sue is an ancestor of Mary and Mary is an ancestor of Anne, then Sue is an ancestor of Anne. Okay, this was not initially into my graph system, but with applying these functional properties, I can then add and augment the system with this extra data.

So that's very useful. Then there's some properties called functional properties, which means only one. So has father is a functional property.

You can only have one father. You can only have one mother. That is a functional property.

So that can serve as a constraint. So if you say Bob is Jim's father, BB is Jim's father, well, the inference here is that Bob and BB are two ways of representing the same individual, because that is a functional property. You can only have one.

So these derivations and constraints don't sit in the graph. They sit sort of on the side, and they can help, as we're going to see, I'm going to propose, when we deal with agents, how they can help us out. So what about agents? Everybody's talking about agents now, and everybody's talking about loops, loops, loops, loops everywhere.

Loops have been around for a long time. Back in the 60s, people were debating who has the best programming language, Fortran or COBOL? No, mine is better. No, mine is better.

Oh, you don't know anything. You don't know what you're talking about. Beam and Giacopini in 1966 came out and said, okay, there is no real difference in programming languages if they have three aspects, sequence.

I can put statement A, statement B, statement C, fine. I have conditionals. I can have if-then.

And the last piece, I have a loop. If I have a loop, if I have iteration, if I take these three things, the language is what's called Turing-complete, can compute anything that can be computed by computational devices from the work of Alan Turing. And now we're seeing this in agentic AI.

Agents now have loops. Loops give us the last piece in the equation of giving us a technology that is capable of doing anything that computational devices can do. The danger, though, of loops is that they can break.

If you're a programmer, you know you all go into an infinite loop. Not good. However, loops can drift.

As agents start talking to each other, things go off the rails. And loops can cost you money. Token counts crank up as the loops continue.

So you need to be careful. But in a way, we are revisiting some of the early stuff with symbolic AI. I would argue we're going back to the world of expert systems.

Which is the symbolic part of the whole thing. So I want to show you an example using Claude agent. So a little code here.

Don't get scared. But I know nobody does Python anymore. But you got to look at what the agent is giving you.

And you got to move in and manipulate it. So here's a loop, while true, classic Python loop. And so we have a client.

So the first little chunk here that you see, RESP, the response. This is just some code where we have a model. And we have a prompt.

That's part of the messages. And we have a tool. And we're asking the LLM to solve this problem using a tool.

Now, here's the catch. LLMs can't do anything. All they can do is give us the next word with a high probability.

Amazingly, we can now have these conversations with it. But they can't do anything. But we can give it a tool.

And we can give it what we want. And say, how do you think this tool can help us get what we want? And then the LLM will set up the parameters. And come back to us and say, OK, here's my response.

I can't execute this tool. But I know what the input parameters are. I know what your context is.

I know what your prompt is. So here is the call that you need to make of the tool. Because I can't do it.

I'm the LLM. I'm just locked in this box. So the second chunk is stop reason.

So stop reason means the LLM has stopped for some reason. The reason here is that it can't do anything. And if the reason is tool use, ah, now it's time.

Let's go execute that tool. So that second line, get tool, it takes the response, which is formulating the parameters and triggering the action. Now, there's stuff in red here.

This is where I think the LLMs and other, I'm sorry, not LLMs, the ontologies and stuff can come in. So if you look down there, after the tool is called, it said tool runs, this is where ontologies could come in. The tool is going to give us information.

We put the information in a form that our validator can use. And think about the validator as operating with these ontologies about our domain. Then we can make some sense of whether the response of the LLM is reasonable.

So this is the loop. Call a tool, check the stop reason. If it's a reasonable result, then let's go with it.

If it's not reasonable, go back to the LLM. Say, oh, this is not working. Or get a human in the loop.

But the idea is to surround the input with checks. Now, I've got this, something that you should be at least taking a look at if you're doing some of this coding, is something called Pydantic. Pydantic is a way to specify the types of, what you want the types of the parameters to be.

Those of you who know Python know Python is a unstructured type language. So you can have a variable, `x equals 20`, `x equals hello`, no problem. There's no typing.

Pydantic adds typing to that. So you want to check your types with Pydantic, and then check your results with the ontology. So Pydantic at the door, ontology at the ledger, and pure agents.

By the way, your agents should try to have no side effects. That helps the whole logic, meaning they're not running off doing something that they're changing. They're changing things in the database.

Not yet. You want to run them through the ontology first and make sure that works. OK, I've got another, just a short time.

I'm going to try to show you some of the things that you can, some logical constructs from something called OWL, the Web Object Language for Objects. So you have these functional properties, disjoint properties. I'll just put these.

You can look at the slides. But essentially, the errors it can catch, look over in the right-hand column. A second refund on the same order is a problem.

Ontologies could catch it, whereas it's very tricky to do that in English. A payout sent to the support desk instead of the buyer. You can catch that with an OWL disjoint property, where customer and support rep are two separate entities.

One of, a made up value like probably shipped, you can specify, must have certain kinds of value. So the status, paid, shipped, or refunded, nothing else. And when you're in the pure text world, this can get funky, because the LLMs are, again, probabilistic and return some crazy stuff.

OK. So really, the point I want to make here is you can have a reasoner built on ontology to check, keep the LLM on track, have guardrails to keep it honest. And for the guardrails, I'm referring to these concepts, these support technologies with RDFS and OWL.

And my bottom line is, nothing is a mistake. There's no win, no fail, only make. OK.

Feel free to reach out to me, Coyle at Berkeley. I've got a website, cosupreme.ai. I'm a big fan of, if you're John Coltrane, has some jazz called Love Supreme. So I've named my site Code Supreme.

And if you go there, I've got some music, and it's all good. OK. Thanks very much.

20 minutes. Thanks.

Transcribed by [TurboScribe](#). [Go Unlimited](#) to remove this message.